What is actually implemented

Tier 5 items 3–5. The LLD describes a design that was partly invented before anyone had seen the code. This is the authoritative map from that vocabulary to the real implementation, with every formula and constant as it exists, so the documents can be corrected against something true.

Numbers here are **not** restated from memory — they come from `results/CANONICAL.md`, regenerated by `make results`.

---

1. The `cetp.*` package now exists

The LLD invented package names (`cetp.profiling`, `cetp.hardware`, `cetp.models`, `cetp.gating`, `cetp.cicd`). Those specific names were never real. A `cetp` package **does** now exist, with a different and smaller shape:

module	what it does
`cetp.predict`	`features_from_plan`, `analytical_scaling`, `compute_only_scaling`, `hardware_distance_confidence`, `predict`
`cetp.regress`	`detect`, `format_report` — regression detection, the recommended mode
`cetp.signature`	signature resolution, the registry, `ratios`, `warnings_for`
`cetp.redact`	`redact_plan`, `assert_clean` — strip literals before a plan leaves the machine
`cetp.cli`	`cetp predict / regress / redact / signature / selftest`

The research code stays in `scripts/` and is not part of the package. The gate (`gate/cetp_gate.py`) is retained unchanged so its 28 tests keep passing, but **it loads the stale pickles** — new work should use `cetp.predict`, which does not.

2. The confidence gate, exactly as implemented

`gate/cetp_gate.py:102` and `cetp/predict.py:hardware_distance_confidence`:

```
normalise each machine's (bandwidth, compute) to [0,1] over the reference set
d          = min over reference machines of euclidean_distance(prod, machine)
confidence = max(0, 1 - d / sqrt(2))      # sqrt(2) is the diagonal of the unit square
low        = confidence < 0.6
```

The 0.6 is a hand-set constant and was never derived. Tier 1 tried to derive it and could not: the score depends only on the production machine, so under leave-one-machine-out it takes **one value per machine — six distinct operating points in total**. Worse, risk does not fall monotonically as it abstains more (90% coverage gives 24.28% MAPE, but 80% gives 25.25% and 70% gives 26.17%).

It is also guarding the wrong component. Split at 0.6:

predictor	MAPE, low confidence	MAPE, high confidence	ratio
learned GBR	38.15%	22.31%	**1.71x**
analytical	18.79%	21.93%	0.86x (**inverted**)
cmp_only	20.10%	20.06%	1.00x (**flat**)

It works on the learned model, is inverted for the analytical formula, and is useless for `cmp_only`. Since the deployment recommendation is a formula, **the gate protects the one component the project does not recommend shipping.** Documented in `results/tier1_gate.md`.

3. The analytical weights, exactly as implemented

`gate/cetp_gate.py:68–71`:

```
inv_bw  = 1 / bandwidth_ratio
inv_cmp = 1 / compute_ratio
scaling =  (p_bandwidth + p_io) * inv_bw          # io is folded in with bandwidth
         +  p_compute           * inv_cmp
         +  p_mixed             * sqrt(inv_bw * inv_cmp)   # geometric mean
```

Two facts the LLD should state:

- **The `io` term is applied to `1/bandwidth_ratio`, not to 1.0.** The LLD claimed `io = 1.0`. This is why `results/CANONICAL.md` §4 shows *no* improvement over the baseline on io-labelled queries — by construction there cannot be one.
- **The weighting contributes nothing measurable.** A flat uniform split scores **21.55%** against **21.61%** for the trained classifier and **21.76%** for measured resource elasticity. `cetp.predict.analytical_scaling` therefore defaults to the **uniform** weighting and needs no classifier at all.

4. Claims to delete from the LLD

LLD claim	Reality
"empirical percentile margins m_95 / m_99"	**Never implemented.** Nothing by that name exists. Percentiles now come from `models/calibration.json`: conformal quantiles of `log(actual/predicted)` computed leave-one-machine-out. The shipped multipliers are p90 ×1.366, p95 ×1.472, p99 ×1.967.
`quantile_regressor.pkl`, `bottleneck_clf.pkl`, `hardware_signatures.json`	Wrong names. The real artifacts are `models/scaling_quantile_models.pkl`, `models/bottleneck_classifier.pkl`, `models/hardware_signature.pkl`. `models/hardware_signatures.json` **now exists** but is the new probe registry, not what the LLD meant.
a `captured_runs` table	The real table is `measurements`, with a `source` column distinguishing `seed` from `captured`. Schema in `scripts/analysis/measurement_store.py`.
`HardwareSignature.ratioTo` / `.distanceTo`	No such class. The logic is `cetp.signature.ratios()` and `cetp.predict.hardware_distance_confidence()`.

5. The hardware signature as implemented

Two paths, and **they are not on the same scale** — `cetp.signature.ratios()` refuses to mix them:

- **legacy (frozen)** — the five EC2 machines. Bandwidth from a STREAM-triad probe **whose source no longer exists**, so these five numbers are unreproducible and no new machine can join this scale. Compute is multi-core aggregate GFLOPS probed at `nproc`.
- **registry (v2)** — `scripts/collection/hw_probe_suite.c`, seven axes, recorded in `models/hardware_signatures.json` with host facts, PostgreSQL settings, a source checksum and a validation verdict. Compute is probed at the **achievable DOP** (`1 + max_parallel_workers_per_gather`), not at `nproc`.

Why the DOP matters: a query can never use more than that many processes, so an `nproc`-aggregate figure describes capacity nothing can reach. On the 8-core laptop the two differ by 1.69× (58.84 vs 34.90 GFLOPS) and the mistake cost **324% vs 153% MAPE** (`results/tier1_laptop_dev.md`).

And the signature is effectively one-dimensional: across the five reference machines its two axes correlate at **r = +0.958**, with **97.9%** of variance on one principal component, and that axis largely tracks core count (r = +0.841). Every prediction should be read as driven by one capability axis, not two (`results/tier1_hw_signature.md`).

6. What the gate verdicts mean

`three_state_verdict` in `gate/cetp_gate.py:120`:

```
WARN   if confidence < 0.6                    -> outlier hardware, measure instead
BLOCK  if predicted p50 > SLA                 -> already over on the point estimate
PASS   if predicted p99 <= SLA                -> clears even the tail
WARN   otherwise                              -> p50 under, p99 over: run a canary
```

Measured behaviour, which the deck overstates: at a scaling threshold of 1.5 this blocks about 8% of pairs with a **54.5% false-alarm rate**, and the PASS state covered **4 of 840** pairs. A conformal replacement delivers its guarantee on PASS (1.37% breach at a nominal 5%) but **can never BLOCK** at these interval widths. The honest description is a two-state gate: *guaranteed-pass* or *defer to a real run*. See `results/tier1_gate.md`.

7. Confirmed correct, safe to claim

- **Root-node-only buffer capture** — `root_buffers()`. Summing PostgreSQL's cumulative per-node counters up the tree was the bug; it inflated the original dataset 3.06×–12.00×.
- **Four-class probability output** feeding the regressor as `p_compute/p_bandwidth/p_io/p_mixed`.
- **Quantile regressors** for p50/p95/p99 with `random_state=42`.
- **13 features, no `prod_*` leakage.** Note six of them (`dev_cost`, `dev_total_buffers`, `dev_shared_hit`, `dev_shared_read`, `dev_rows`, `dev_io_ratio`) are **machine-invariant on the corrected dataset**, i.e. a one-hot encoding of `query_id` in disguise.
- **CI/CD gate** returns PASS/BLOCK/WARN and runs in `.github/workflows/` on every push.
- **Self-calibration loop** — persist, register machine, retrain, versioned into `models_online/v<N>/`.
- **22 queries**, q15 restored via a `NOT MATERIALIZED` CTE that reproduces the spec view form's plan exactly (`results/q15_resolution.md`).